

Fundamentals of Data Science

Lecture 8 - Evaluation and Interpretability

Adrian Ochs ¹ and Christian Roerig ²

Michaelmas 2025

¹Co-founder Quantum Leap, ex-QuantCo

²Data Scientist/ML Engineer @ QuantCo

Table of contents

1. Importance of interpretability
2. Taxonomy of interpretation methods
3. Python libraries for interpretability
4. Global model-agnostic methods
 - Partial dependence plots
 - Accumulated local effect plots (ALE)
 - Feature interaction - H2-statistic
5. Local model-agnostic methods
 - Individual Conditional Expectation
 - Shapley values
6. Explainable Boosting Machine

Importance of interpretability

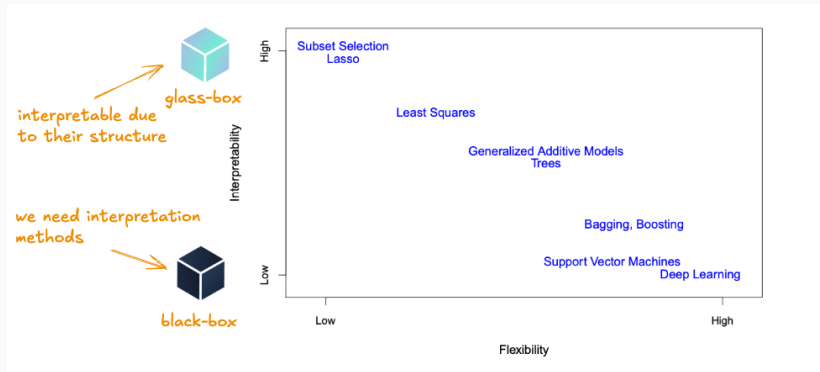
Importance of interpretability

Making models interpretable and accessible for developers, data scientists and business stakeholders helps with (acc. to [InterpretML](#)):

- **Model debugging** - Why did my model make this mistake?
- **Feature Engineering** - How can I improve my model?
- **Detecting fairness issues** - Does my model discriminate?
- **Human-AI cooperation** - How can I understand and trust the model's decisions?
- **Regulatory compliance** - Does my model satisfy legal requirements?
- **High-risk applications** - Healthcare, finance, judicial, ...

Importance of interpretability

Similar to modelling constraints, we might face implicit constraints on the interpretability of the model. This could be due to tech stack (e.g. can only deploy parametrized models), user needs (model should be editable) etc.



Taxonomy of interpretation methods

Taxonomy of interpretation methods

Intrinsic interpretability or post hoc: We distinguish models which are intrinsically interpretable due to simple structures and such for which we need tooling.

Model-specific vs. model-agnostic: Model-specific tools are limited to a class of models (e.g. interpreting coefficients in parametric models), whereas model-agnostic methods can be applied to any model producing predictions based on features.

Local vs. global: Global methods explain the general behaviour of the model, while local methods decompose individual predictions.

Python libraries for interpretability

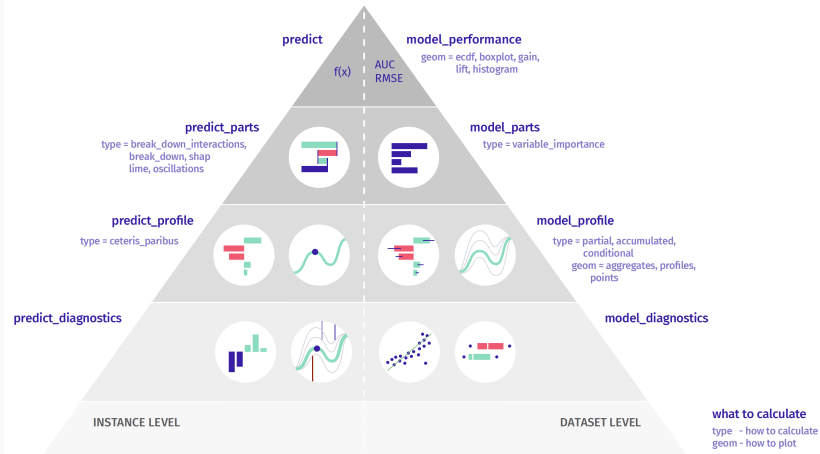
Python libraries for interpretability

- **DALEX** (robot): moDeL Agnostic Language for Exploration and eXplanation
- **InterpretML**: InterpretML is an open-source package that incorporates state-of-the-art machine learning interpretability
- **scikit-learn**: Scikit-learn offers some basic interpretability tools (e.g. PDPs)
- **SHAP**: A game theoretic approach to explain the output of any machine learning model.



Example: DALEX

DALEX: moDEL Agnostic Language for Exploration and eXplanation



Global model-agnostic methods

Global model-agnostic methods

Global methods describe the average behavior of a machine learning model. The counterpart to global methods are local methods. Global methods are often expressed as expected values based on the distribution of the data (see. [Chapter 8, Interpretable ML](#)).

We will cover:

- **Partial dependence plots:** Illustrates the average feature effect marginalizing over other features.
- **Cumulated local effect plots (ALE):** Corrects feature effect for feature dependence.
- **Feature interaction (H2-statistic):** Quantifies to what extent the prediction is driven by joint effects of features.

Partial dependence plots

Partial dependence plots were introduced by [Friedman \(1999\)](#) in the context of GBMs. They show the marginal effect of one (univariate PDP) or two (bivariate PDP) features on the prediction.

The partial dependence function for regression is defined as:

$$\hat{f}_s(x_s) = E_{X_C}[\hat{f}_s(x_s, X_C)] = \int \hat{f}_s(x_s) dP(X_C),$$

where:

- x_s : feature for which PDP is computed
- x_C : all other features, held constant

Hence, we marginalize over all other features and get a function which only depends on the features in S we are interested in.

Partial dependence plots

This translates to computing the sample mean for various values of x_s , while keeping x_C fixed:

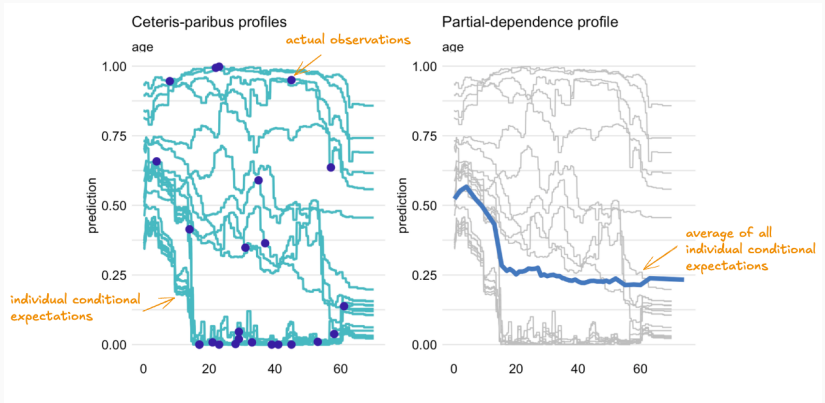
$$\hat{f}_s(x_s) = \frac{1}{n} \sum_{i=1}^n \hat{f}_s(x_s, x_C^i)$$

With correlated features this is likely to produce combinations of features which are very unlikely or even impossible. Yet, it gives us the average "mechanical" effect of changes in x_s on the predicted outcomes.

Partial dependence plots - recipe

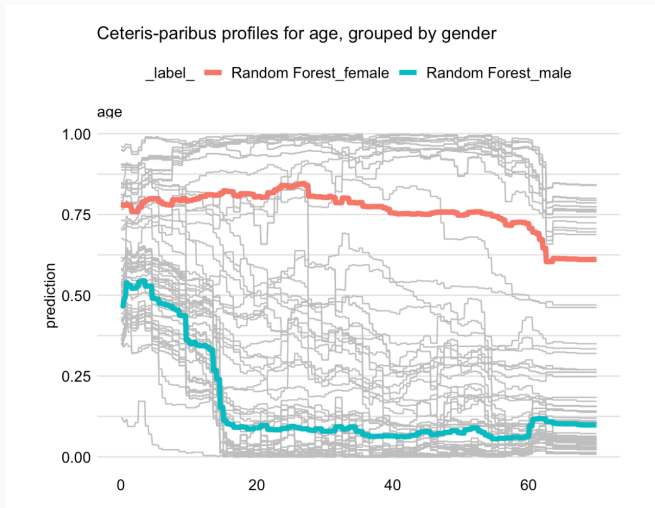
1. Select feature x_s
2. Define grid for feature (numeric or categorical) on domain of x_s
3. Iterate over grid:
 - 3.1 Set grid value for feature
 - 3.2 Compute and average predictions
4. Plot average predictions over grid

Partial dependence plots - examples



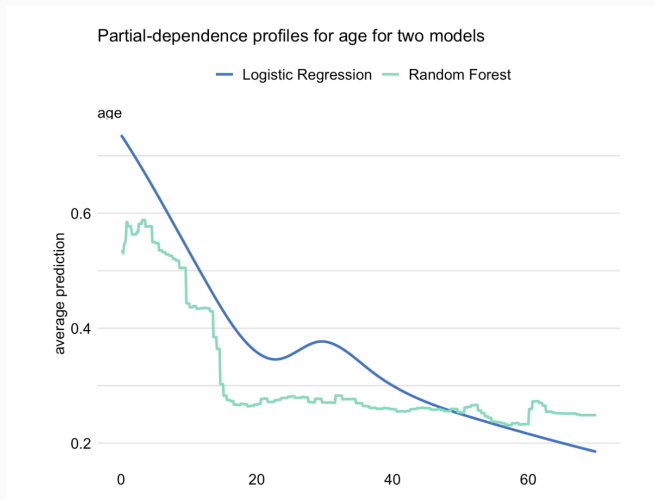
Source: EMA Chapter 17

Partial dependence plots - examples



Source: [EMA Chapter 17](#)

Partial dependence plots - examples



Source: [EMA Chapter 17](#)

PDPs using various packages - Dalex

```
import dalex as dx
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import make_column_transformer
from sklearn.pipeline import make_pipeline

titanic = dx.datasets.load_titanic()
X = titanic.drop(columns='survived')
y = titanic.survived

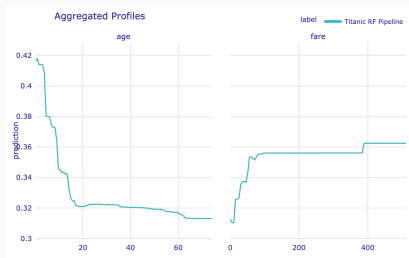
preprocess = make_column_transformer(
    (StandardScaler(), ['age', 'fare', 'parch', 'sibsp']),
    (OneHotEncoder(), ['gender', 'class', 'embarked']))

titanic_rf = make_pipeline(
    preprocess,
    RandomForestClassifier(max_depth = 3, n_estimators = 500))
titanic_rf.fit(X, y)

titanic_rf_exp = dx.Explainer(titanic_rf, X, y,
                              label = "Titanic RF Pipeline", verbose=0)

pd_rf = titanic_rf_exp.model_profile(variables = ['age', 'fare'])
pd_rf.plot()
```

✓ 1.1s



PDPs using various packages - Scikit-learn

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import make_column_transformer
from sklearn.inspection import PartialDependenceDisplay
from sklearn.pipeline import make_pipeline

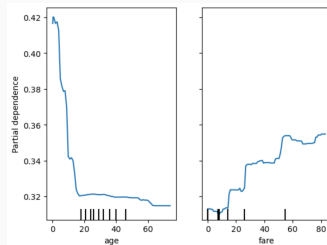
titanic = dx.datasets.load_titanic()
X = titanic.drop(columns='survived')
y = titanic.survived

preprocess = make_column_transformer(
    (StandardScaler(), ['age', 'fare', 'parch', 'sibsp']),
    (OneHotEncoder(), ['gender', 'class', 'embarked']))

titanic_rf = make_pipeline(
    preprocess,
    RandomForestClassifier(max_depth = 3, n_estimators = 500))
titanic_rf.fit(X, y)

PartialDependenceDisplay.from_estimator(titanic_rf, X, features=['age', 'fare'])
```

✓ 6.8s



Advantages:

- Simple and intuitive way to summarize feature effect.

Disadvantages:

- For correlated features they are biased as they assume independence.
- For large datasets expensive to compute. Yet, quite robust to sampling.

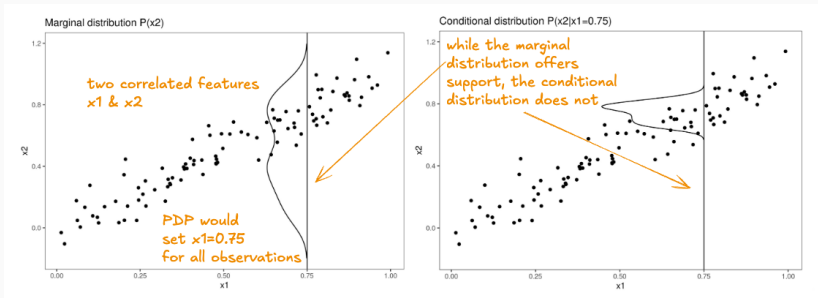
Accumulated local effect plots (ALE)

Accumulated local effects aim to address one drawback of PDPs by accounting for potential correlation between features. ALE plots were proposed by [Apley \(2019\)](#).

Example

In many applications correlated features are the norm. Take the example of house price predictions. The surface and the number of rooms will be positively correlated, and a surface of 15 square meters with 5 rooms, does not make sense.

ALE - Motivation

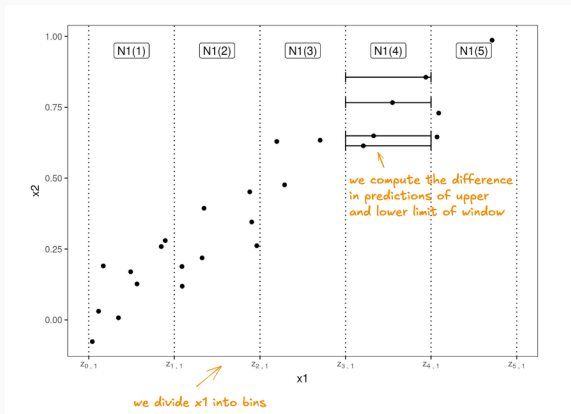


Source: [Interpretable ML Chapter 8](#)

Instead of computing average predictions, we could compute conditional average predictions. This is known as **Marginal Plots**, however they measure the combined effect of the features due to the correlation. Hence, how can we disentangle the feature effects, while still accounting for correlation? This is what ALE plots aim to achieve.

ALE - Method details

Instead of simply computing conditional averages, ALE plots compute differences in predictions, while conditioning on other features. Essentially, it shows us how model predictions change in a small window around the grid point for all data instances falling in that window.



Recall the PDP definition:

$$\hat{f}_{s,PDP}(x_s) = E_{X_C}[\hat{f}_s(x_s, X_C)] = \int \hat{f}_s(x_s) dP(X_C),$$

when taking the conditional distribution, we get the definition for M-plots:

$$\hat{f}_{s,M}(x_s) = E_{X_C, X_S}[\hat{f}_s(x_s, X_C) | X_S = x_s] = \int \hat{f}_s(x_s) dP(X_C | X_S = x_s),$$

ALE - Method details

$$\begin{aligned}\hat{f}_{S,ALE}(x_S) &= \int_{z_{0,S}}^{x_S} E_{X_C|X_S=x_S} \left[\hat{f}^S(X_s, X_c) | X_S = z_S \right] dz_S - \text{constant} \\ &= \int_{z_{0,S}}^{x_S} \left(\int_{x_C} \hat{f}^S(z_s, X_c) d\mathbb{P}(X_C | X_S = z_S) \right) dz_S - \text{constant}\end{aligned}$$

Annotations:

- partial derivative of prediction w.r.t. feature s (points to $\hat{f}^S$)
- integration over all partial derivatives (points to dz_S)
- some centering (points to the inner integral)

ALE - Method details

$$\begin{aligned}\hat{f}_{S,ALE}(x_S) &= \int_{z_{0,S}}^{x_S} E_{X_C|X_S=x_S} \left[\hat{f}^S(X_S, X_C) | X_S = z_S \right] dz_S - \text{constant} \\ &= \int_{z_{0,S}}^{x_S} \left(\int_{x_C} \hat{f}^S(z_S, X_C) d\mathbb{P}(X_C | X_S = z_S) \right) dz_S - \text{constant}\end{aligned}$$

Annotations:

- partial derivative of prediction w.r.t. feature s (points to $\hat{f}^S$)
- integration over all partial derivatives (points to dz_S)
- some centering (points to the constant term)

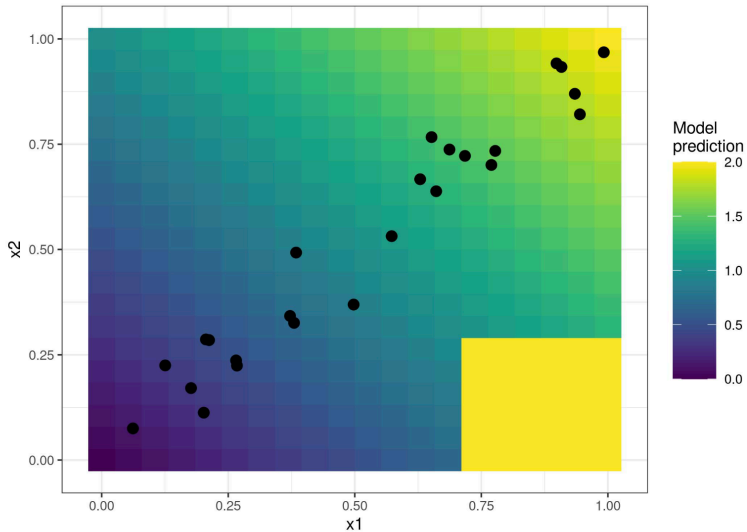
The discretized empirical version simply considers the average difference between all bins:

$$\hat{\tilde{f}}_{j,ALE}(x) = \sum_{k=1}^{k_j(x)} \frac{1}{n_j(k)} \sum_{i: x_j^{(i)} \in N_j(k)} \left[\hat{f}(z_{k,j}, x_{-j}^{(i)}) - \hat{f}(z_{k-1,j}, x_{-j}^{(i)}) \right]$$

The measured effect is centred so that the mean effect is zero:

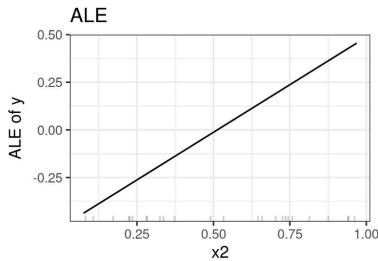
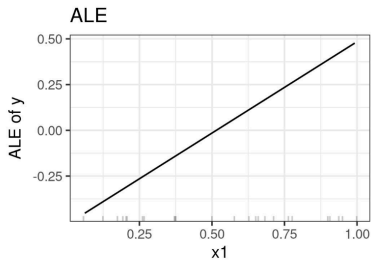
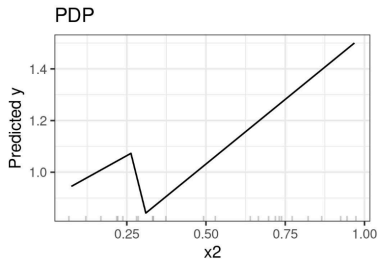
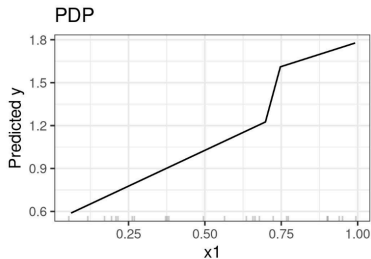
$$\hat{f}_{j,ALE}(x) = \hat{\tilde{f}}_{j,ALE}(x) - \frac{1}{n} \sum_{i=1}^n \hat{\tilde{f}}_{j,ALE}(x_j^{(i)})$$

ALE vs. PDP



Source: [Interpretable ML Chapter 8](#)

ALE vs. PDP



Source: [Interpretable ML Chapter 8](#)

Advantages:

- Unlike partial dependence plots (PDP), ALE plots are not biased by correlations between features.
- Generally faster to compute compared to other model interpretation techniques, especially for large datasets.

Disadvantages:

- Interpretation is more complex due to conditional difference computation.
- Sensitivity to the specific binning.

One way to measure the interaction strength is to measure how much variation of the prediction depends on the interaction of the features. This is known as H2-statistic by [Friedman & Popescu \(2008\)](#).

If two features do not interact, we can decompose the PDP simply by summing the univariate PDPs:

$$PD_{jk}(x_j, x_k) = PD_j(x_j) + PD_k(x_k).$$

Feature interaction - H2-statistic

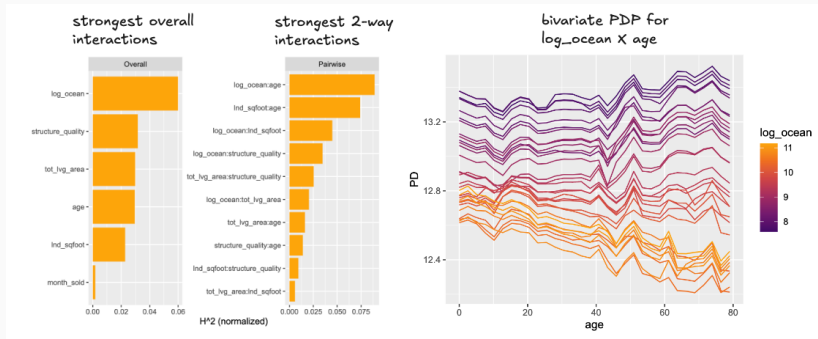
Now, we want a test statistic which is 0, if there is no interaction, i.e. if the bivariate PD, is simply the sum of the univariate PDs, and which is 1, if the variation of the prediction is purely coming from the interaction.

This is the H2-statistic:

$$H_{jk}^2 = \frac{\sum_{i=1}^n (PD_{jk}(x_j^i, x_k^i) - PD_j(x_j^i) - PD_k(x_k^i))^2}{\sum_{i=1}^n PD_{jk}^2(x_j^i, x_k^i)}$$

Feature interaction - H2-statistic - examples

For implementation, see: [R-package hstats](#).



Advantages:

- Underlying theory through partial dependence decomposition.
- Meaningful interpretation: Share of variance explained by interaction.
- Dimensionless, so it can be compared across features and models.

Disadvantages:

- Computationally expensive.

Local model-agnostic methods

So far we have considered methods which are explaining the overall model behaviour. Local methods explain individual predictions.

What we will cover:

- Individual conditional explanation curves (ICE)
- Shapley values

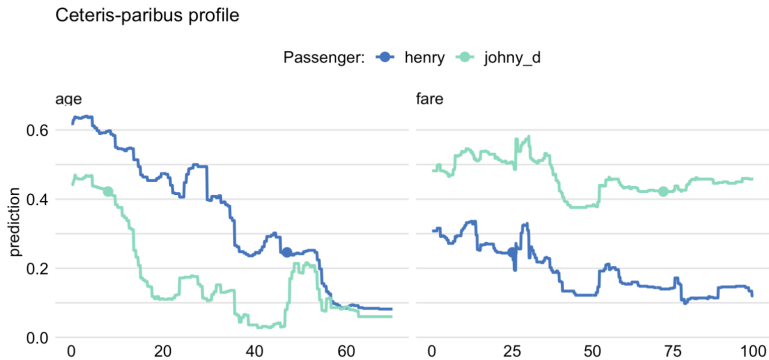
Individual Conditional Expectation

Individual Conditional Expectation plots (also sometimes referred to as Ceteris Paribus Profiles) are the building block for PDPs, illustrating how predictions for specific observations change, if only one feature is changed. The same drawbacks as for PDPs apply.

Instead of taking the mean over all observations, we simply consider the predictions over the grid x_s , for an individual observation i :

$$\hat{f}_s(x_s)^i = \hat{f}_s(x_s, x_C^i)$$

The advantage relative to PDPs is, is that we get a better **view on the heterogeneity of predictions**.



We often face the scenario that we have a high-dimensional model with many interactions and the question is how much does each feature contribute to the specific prediction.

Take a simple linear model:

$$y = 0.5x_1 + 0.2x_2,$$

here the contributions are simply the model weights and the values of x_1, x_2 . Yet, adding an interaction x_1x_2 makes the interpretation harder and we have to decide on how to decompose the interaction effect into additive contributions of x_1 and x_2 .

Shapley values

The theory behind Shapley values in machine learning originates from cooperative game theory. Features can be interpreted as players and predictions are the payout. The problem to solve is: **How to distribute the model's prediction across features?**

The Shapley value has the following properties:

- **Efficiency:** The feature contributions must add up to the difference of the specific prediction and the average:

$$\sum_{j=1}^p \phi_j = \hat{f}(x) - E(\hat{f}(X)).$$

- **Symmetry:** If features contribute equally, then $\phi_i = \phi_j$.
- **Dummy:** A feature that does not change the predicted value it should have Shapley value of 0: $\phi_j = 0$.
- **Additivity:** If predictions are additive, so are Shapley values (e.g. multiple trees).

Shapley values

Let's start simple. What are the Shapley values in a simple linear model?
In the context of GLMs, we would compute Shapley values based on the linear predictor, before applying the link function.

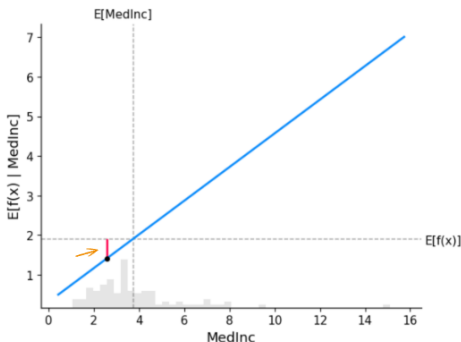
```
print("Model coefficients:\n")
for i in range(X.shape[1]):
    print(X.columns[i], "=", model.coef_[i].round(5))
✓ 0.0s
```

Model coefficients:

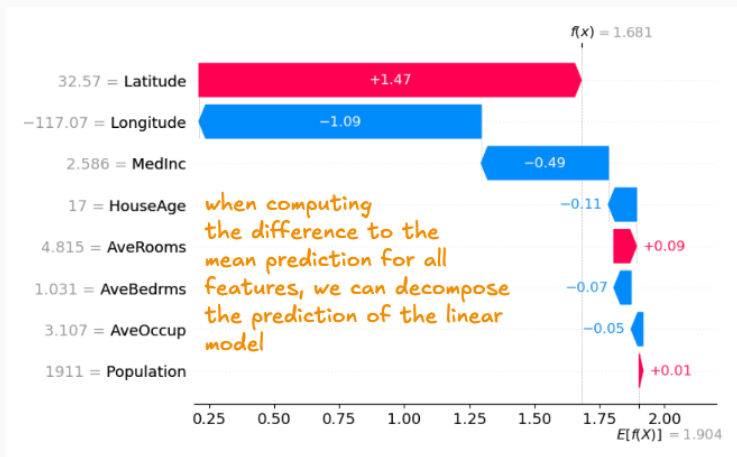
MedInc	= 0.42563
HouseAge	= 0.01033
AveRooms	= -0.1161
AveBedrms	= 0.66385
Population	= 3e-05
AveOccup	= -0.26006
Latitude	= -0.46734
Longitude	= -0.46272

we can't interpret coefficients directly as contributions as the underlying scales are different

yet, in the simple linear model, we can read the Shapley value directly from the PDP, as the difference of the prediction at $\text{MedInc} \sim 2.5$ relative to the mean

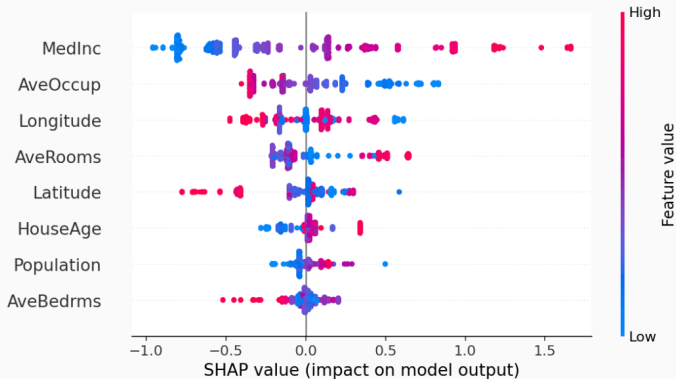


Shapley values



Shapley values

When plotting Shapley values for multiple observations, we get a so-called beeswarm plot, which indicates which features have the largest impact on the predictions for given feature values:



Shapley values

Yet, how do we attribute feature effects if we have a more complex models? The good news is that Shapley values offer a unique solution to the decomposition problem.

The diagram shows the Shapley value formula for feature i :
$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} [f_{S \cup \{i\}}(x_{S \cup \{i\}}) - f_S(x_S)]$$

Annotations with arrows:
- ϕ_i : Shapley value of feature i
- $S \subseteq F \setminus \{i\}$: subset of all features (pointing to S) and set of all features (pointing to F)
- $|S|!(|F| - |S| - 1)! / |F|!$: subset combinations (pointing to the denominator)
- $f_{S \cup \{i\}}(x_{S \cup \{i\}})$: model with feature i
- $f_S(x_S)$: model w/o feature i

The Shapley value of a specific feature is the weighted average difference of models including the feature and models without the feature over all combinations of possible subsets.

The actual computation would be NP-hard as we would have to fit many models of all possible combinations of subsets of features.

Luckily good approximations exist: [Strumbelj & Kononenko \(2024\)](#) & [Lundberg & Lee \(2017\)](#), with Lundberg being the author of [SHAP](#).

More details on KernelSHAP and TreeSHAP: [Interpretable ML Chapter 9](#).

Explainable Boosting Machine

While linear models offer great interpretability, we have to manually parametrize them and often times their performance is worse than e.g. GBMs. Yet, we can restrict GBMs in a way that we get the best of both worlds: a linear model structure and good performance through automatic feature engineering.

This class of models was coined **Explainable Boosting Machine** by [Lou et al. \(2013\)](#).

EBM is a generalized additive model (GAM) of the form:

$$E[y] = g^{-1}(\beta_0 + \sum f_j(x_j))$$

The advantage over traditional GAMs is that the feature function f_j is learned by gradient boosting. However, in contrast to general GBMs, the boosting is restricted to train on one feature at a time and to include at most two-way interactions between the features.

Recall that in when using standard decision trees (aka no linear trees) as learners, we get piecewise-constant indicator functions in the leaves.

Hence, we can also collect all those splits and node weights and construct essentially a coefficient table where the split conditions can be expressed as indicators/dummies.

Molnar, C. (2022). Interpretable Machine Learning: A Guide for Making Black Box Models Explainable (2nd ed.). [Link](#)

Dalex: <https://dalex.drwhy.ai/python/>

InterpretML: <https://interpret.ml/>

SHAP: <https://shap.readthedocs.io/en/latest/index.html>

Exploratory Model Analysis: <https://ema.drwhy.ai/>